Prompt Log: Lab Color Sorter (ml5.js)

This file documents the prompts used to finish the game, as required by the README's "What to Submit" section. For each prompt it gives the prompt text, a one-sentence purpose, where the project was before and after, and how the result was validated.

AI tool: Claude (Anthropic), used in the claude.ai chat interface. The whole project was uploaded as a zip, and Claude read the files before suggesting or making changes.

Working method: one prompt at a time, in the order of the README's "Example Prompts". After each prompt I applied the change, tested it in the browser, and reported back before the next prompt was run.

Scope: the only file changed in the whole project is `js/ML.js`. The rest of the game (`index.html`, `js/script.js`, `js/ColorGame.js`, `js/colorData.js`) was left exactly as provided.

Prompt 1: Read the project and explain what the two methods need to do

Prompt (my chat messages, lightly trimmed):

> Please scan the zipped project first, understand the intent of the code before making any changes.
> Our goal is to finish the missing logic inside the two helper methods within js/ML.js. Step 1. Describe. What is the helper method used for? Where is it invoked? What references would it need to function as intended? Analyze both `async trainModel(updateProgress) {}` and `async classify(color) {}`. (Don't forget to preview the logic of the index.html to see how the user input functions.)

This matches the README's first example prompt:

```
Read the starter files and explain what `trainModel()` and `classify(color)`
need to do for the game to use a trained model. Do not change any files.
```

Purpose: to understand how the game uses the model before writing any code.

Before: `js/ML.js` contained the import of `COLORS` and `createColorExample`, the constants `TRAINING_EXAMPLES_PER_COLOR` (40) and `TRAINING_OPTIONS` (`epochs: 40`, `batchSize: 12`), and an `ML` class whose `trainModel(updateProgress)` and `classify(color)` methods were empty.

After: no files changed. The analysis established these points:

- **Not a genetic algorithm.** The project trains a small neural network with `ml5.neuralNetwork`. There is no population, mutation, or selection.
- **`trainModel`:** it is called from the Train button handler in `js/script.js`. It must not resolve until training is finished, it must let errors reach the caller (the handler shows `error.message`), and it must store the model on the instance because

`game.setClassifier(ml)` uses the `ML` object itself as the classifier. It takes a status string callback (`updateProgress`) that is written into the status paragraph.

- **`classify`:** it is called once per swatch from `ColorGame.checkResults()` via `Promise.all`, so 10 calls run at once. It receives the whole swatch object, which includes the true `label`, so it must read only `red`, `green`, and `blue`. It must return a bare lowercase string because the game compares it to `color.label` with `===`, and it must throw a clear error if no model exists.
- **Needed references:** `COLORS`, `createColorExample`, `TRAINING_EXAMPLES_PER_COLOR`, `TRAINING_OPTIONS`, and the global `ml5` loaded by the script tag in `index.html`.
- **ml5 sequence:** `ml5.neuralNetwork({ task: 'classification' })`, then `addData`, `normalizeData`, `train`, and finally `classify`.

Prompt 2: Create the training-data helper

Prompt (README example prompt 2):

```
In `js/ML.js`, create a helper that returns labeled RGB training examples for
red, green, and blue. Return an array of objects with `red`, `green`, `blue`,
and `label` properties. Make 40 examples for each color. Only change `js/ML.js`.
```

Purpose: to produce the labeled dataset that `trainModel` will train on.

Before: the constants and imports existed but nothing used them, so there was no training data.

After: `js/ML.js` has a module-level `createTrainingExamples()` function. It loops over `COLORS` with `flatMap`, calls `createColorExample` 40 times per color using `Array.from`, and returns 120 objects shaped `{ red, green, blue, label }`. Nothing called it yet, so game behavior was unchanged.

Validation:

- Claude ran it in Node against the project's own `colorData.js`. It returned 120 examples with 40 per label, every object had exactly the four expected properties, and all channel values were integers from 0 to 255.
- In the browser, I temporarily logged the function's result. The console showed `Array(120)` with red examples first (indexes 0 to 39), then green (40 to 79), then blue (80 to 119).

Supporting prompt: Analyze a console warning

Prompt (my chat message): I pasted a console message and asked whether it affected the project:

```
Source map error: Error: NetworkError when attempting to fetch resource.
Resource URL: https://unpkg.com/ml5@1/dist/ml5.js
Source Map URL: ml5.js.map
```

Purpose: to decide whether the warning was a real problem before moving on.

Before: an unexplained warning appeared in the console after the helper was added.

After: the warning was identified as harmless. Browser dev tools look for a `.map` file to show readable source for debugging, and fetching the one for ml5 failed. The ml5 script itself had loaded (the warning names it as the resource), so the game was not affected. No code changed.

Prompt 3: Implement `trainModel(updateProgress)`

Prompt (README example prompt 3):

```
Implement `trainModel(updateProgress)` in `js/ML.js`. It should use the training
examples to train a classifier that predicts a color label from RGB values. Save
the trained model on the instance. Call `updateProgress` with status messages
and finish only after training is complete. If training fails, let the error
reach the caller. Use the ml5 documentation linked below. Only change
`js/ML.js`.
```

Purpose: to train a classifier from the 120 examples and save it on the instance so the game can use it.

Before: `trainModel` was an empty method, so clicking **Train model** did nothing useful and the model could never become ready.

After:

- A constructor sets `this.model = null`.
- `trainModel` throws a clear error if `ml5` is not loaded.
- It clears any old model, reports "Creating training data...", builds `ml5.neuralNetwork({ task: 'classification' })`, adds each example as `[red, green, blue]` with `[label]`, and calls `normalizeData()`.
- It then reports "Training model..." and wraps `model.train(options, whileTraining, finished)` in a Promise, so the method resolves only when ml5's finished callback fires. The `whileTraining` callback reports "epoch N of 40".
- Errors, including a rejected promise from `train`, propagate to the caller. `this.model` is set only after training succeeds.

Validation:

- Claude tested it in Node against a mock of the ml5 API (real ml5 could not be loaded there). The mock confirmed the right number and shape of added examples, that the data was normalized, that the promise waited for the finished callback, that progress messages arrived, that `updateProgress` could be omitted, that a sync throw and an async rejection both propagated, and that a failed retrain left the model null.
- In the browser, training completed without errors, and afterward the Train button was disabled (checked with the console command `document.querySelector('#train-button')`), which

is what the click handler does on success.

Prompt 4: Implement `classify(color)`

Prompt (README example prompt 4):

```
Implement `classify(color)` in `js/ML.js`. Given an object with `red`, `green`,
and `blue` values, use the trained model to return the predicted color label as
a lowercase string. Report a clear error if the model has not been trained. Only
change `js/ML.js`.
```

Purpose: to let the game ask the trained model for its predicted label of any swatch.

Before: `classify` was an empty method, so the game's **Check with AI** step had no predictions to compare against the true labels.

After:

- `classify` throws `The model has not been trained yet. Train the model first.` if `this.model` is missing. The game displays this message.
- It reads only `red`, `green`, and `blue` from the object, so the true `label` is never passed to the model, and it throws a clear error if the values are not numbers.
- It wraps ml5's callback in a Promise using the ml5 v1 callback order, `(results, error)`, takes the first (highest confidence) result, and returns its label in lowercase.
- It keeps no shared state, so it is safe for the game's 10 concurrent calls.

Validation:

- Claude tested it in Node against a mock model with game-shaped swatch objects called concurrently through `Promise.all`. The results were valid labels in the right order, the model received only three numbers, an uppercase label was lowercased, and each failure type (untrained, sync throw, rejected promise, error passed to the callback, empty result, bad input) produced a clear error.
- In the browser (`index.html`), I confirmed the game worked end to end: the model's and my scores were computed, deliberately misplacing swatches produced a worse score for me, and **Next round** generated new swatches each time.

Prompt 5: Review `js/ML.js` and explain how to test

Prompt (README example prompt 5):

```
Review `js/ML.js` and check that `trainModel(updateProgress)` and
`classify(color)` meet the requirements above and work with the existing game.
Fix only problems in `js/ML.js`, then tell me how to test the game.
```

Purpose: to check the finished code against every requirement and against how the rest of the game calls it.

Before: both methods were implemented and had passed my browser testing, but the whole file had not been reviewed against the requirements as a unit.

After: no changes were needed, so the file stayed as it was after Prompt 4. The review checked each requirement (40 examples per color, model saved on the instance, progress messages, finishing only after training, errors reaching the caller, lowercase label, clear untrained error, only `js/ML.js` modified). Claude also ran the real `ColorGame` logic against `ML.js` with a mocked ml5:

- Checking before training showed the game's own "Train the model" message.
- A perfect player scored 10/10, and a player with 3 mistakes scored 7/10.
- The check button re-enabled afterward.
- An untrained model wired in directly showed "The model could not classify the colors: The model has not been trained yet. Train the model first."

The review also produced a manual test checklist: serve the page locally, try Check before training, train and watch the status messages, try Check with an unplaced swatch, play a normal round, make deliberate mistakes, start a new round, and block `unpkg.com` to see the failure message.

Final state

- **Changed file:** `js/ML.js` only (helper `createTrainingExamples`, constructor, `trainModel`, `classify`).
- **Game behavior:** the user sorts 10 swatches into red, green, and blue bins, clicks **Check with AI**, and sees both their own score and the model's score out of 10. The model is a small neural network trained in the browser on 120 generated examples.

Notes and observations

- **Mock testing:** the AI's sandbox had no network, so its Node tests used a mock of the ml5 API. All real-ml5 behavior (training finishing, status messages, callback order, scoring) was confirmed by my own testing in the browser.
- **ml5 callback order:** the code uses the ml5 v1 order `(results, error)`, taken from the official v1 color-classifier example. Older ml5 0.x used error-first callbacks, so this matters if the library version in `index.html` is ever changed.
- **Unfixed issue outside `js/ML.js`:** `ColorGame.createSwatch` puts each swatch's true label in its `aria-label` (for example, "RGB 241, 91, 64; red color"), so anyone inspecting the page could read the answers. It does not affect the model, and it was left alone because the assignment limits changes to `js/ML.js`.